

Python

- History and Overview
- Installing Python and Setting up IDEs (e.g., VS Code, PyCharm, Jupyter Notebook)
- Writing and Running Your First Python Program
- Python Syntax and Indentation
- Python Comments

Basic Python Concepts

1. Variables and Data Types

- Numbers (int, float, complex)
- Strings
- Booleans
- NoneType

2. Operators

- Arithmetic, Assignment, Comparison
- Logical, Bitwise, Membership, Identity

3. Input and Output

- Input from User
- Printing Outputs
- String Formatting (f-strings, format(), % operator)

4. Control Flow

- If-Else Statements
- Loops: For and While
- Break, Continue, Pass

Intermediate Python Concepts

1. Data Structures

- Lists: Creation, Methods, Comprehensions
- Tuples: Immutable Sequences
- Sets: Unique Elements and Operations
- Dictionaries: Key-Value Pairs, Comprehensions

2. Functions

- Defining Functions
- Arguments: Positional, Keyword, Default, Arbitrary
- Return Statement
- Lambda Functions
- Recursion
- Decorators

3. Modules and Packages

- Importing Built-in Modules
- Creating and Importing Custom Modules
- Using `pip` to Install External Libraries

4. File Handling

- Reading and Writing Text Files
- Working with CSV and JSON
- Exception Handling in File Operations

Advanced Python Concepts

1. Object-Oriented Programming (OOP)

- Classes and Objects
- Inheritance
- Polymorphism
- Encapsulation
- Magic/Dunder Methods (e.g., `__init__`, `__str__`, `__repr__`)

2. Error and Exception Handling

- Try, Except, Else, Finally
- Custom Exceptions

3. Iterators and Generators

- Creating and Using Iterators
- Using `yield` in Generators

4. Comprehensions

- Nested Comprehensions
- Set and Dictionary Comprehensions

Specialized Topics

1. Regular Expressions

- Pattern Matching
 - Search, Match, and Replace
- 2. **Multithreading and Multiprocessing**
 - Threading Module
 - Global Interpreter Lock (GIL)
 - Multiprocessing Module
- 3. **Networking**
 - Sockets
 - HTTP Requests with `requests` Library
- 4. **Working with Databases**
 - SQLite and `sqlite3` Module
 - CRUD Operations

Data Science and Visualization

1. **NumPy**: Arrays and Linear Algebra Operations
2. **Pandas**: DataFrames, Series, Data Cleaning
3. **Matplotlib**: Basic and Advanced Plotting
4. **Seaborn**: Statistical Visualization

Web Development

1. **Flask/Django**: Creating Web Applications
2. **APIs**: Building and Consuming REST APIs
3. **Web Scraping**: Using `BeautifulSoup` and `Scrapy`

Testing and Debugging

1. **Unit Testing**: Using `unittest`
2. **Debugging Tools**: `pdb` and IDE Debugging Features
3. **Code Profiling**: Using `cProfile`

Advanced Topics

1. **Machine Learning**
 - Using `scikit-learn`
 - Basic ML Models (Linear Regression, Classification)

2. Automation

- Automating Tasks with `selenium` and `pyautogui`

3. Packaging and Deployment

- Creating Python Packages
- Virtual Environments and Dependency Management

Project Work

- **Beginner Projects**
 - Calculator, To-Do List, Guess the Number Game
- **Intermediate Projects**
 - Web Scraper, Chatbot, Personal Expense Tracker
- **Advanced Projects**
 - Full-Stack Web Application, Machine Learning Model Deployment, Automation Tools

Keras

1.1 What is Keras?

- **Overview of Keras:** Keras as a high-level deep learning framework.
- **Integration with TensorFlow:** How Keras is integrated with TensorFlow.
- **Keras Advantages:** Easy model building, flexibility, and scalability.
- **Installing Keras:**
 - Installation using `pip install keras`.
 - Installing Keras with TensorFlow.

1.2 Keras Basics and Architecture

- **Keras Functionalities:** Building, training, evaluating, and deploying deep learning models.
 - **Model Building in Keras:** Sequential API and Functional API.
-

2. Basic Concepts in Keras

2.1 Tensors and Keras Layers

- **Tensors in Keras:** Understanding the data format (Numpy arrays, TensorFlow tensors).
- **Keras Layers:**
 - **Dense Layer:** Fully connected layer.
 - **Activation Layer:** Adding activation functions like ReLU, Sigmoid, etc.
 - **Dropout Layer:** For regularization and reducing overfitting.
 - **Flatten Layer:** Flattening multi-dimensional data.
 - **Reshape and Resizing Layers:** Changing data shapes for model input.

2.2 Keras Models

- **Sequential Model:** Simple stack of layers.
 - **Functional API:** More complex models with multiple inputs/outputs, shared layers.
 - **Model Compilation:**
 - Choosing optimizers (e.g., Adam, SGD).
 - Loss functions (e.g., categorical cross-entropy, MSE).
 - Metrics (e.g., accuracy, precision).
-

3. Building Deep Learning Models with Keras

3.1 Building Your First Model

- **Sequential API:** Building a simple neural network model.
- **Compiling the Model:** Setting up optimizers, loss functions, and metrics.
- **Training the Model:** Using `.fit()` to train models on data.
- **Evaluating the Model:** Using `.evaluate()` to check model performance.

3.2 Model Customization

- **Custom Layers:** Writing your own layer.
- **Functional API for Advanced Architectures:** Building models with shared layers, multiple inputs/outputs.
- **Model Summary:** Understanding and displaying model architecture using `.summary()`.

3.3 Advanced Model Architectures

- **Multiple Inputs and Outputs:** Building models that take multiple inputs or produce multiple outputs.

- **Residual Networks (ResNet):** Creating deep networks with skip connections.
 - **Concatenation Layer:** Merging models for parallel data paths.
-

4. Model Training in Keras

4.1 Training Techniques

- **Batch vs. Mini-batch Training:** Understanding the impact of batch size on model performance.
- **Validation Data:** Using a validation split to monitor overfitting.
- **Epochs and Iterations:** How many epochs to run, and how the data is split.

4.2 Keras Callbacks

- **Early Stopping:** Stop training once the model performance stops improving.
- **ModelCheckpoint:** Saving the model at certain intervals.
- **ReduceLROnPlateau:** Reducing learning rate when the validation loss plateaus.

4.3 Hyperparameter Tuning

- **Grid Search:** Trying out different combinations of hyperparameters.
 - **Random Search:** A randomized method for searching hyperparameters.
 - **Keras Tuner:** An automated hyperparameter optimization framework.
-

5. Regularization and Optimization in Keras

5.1 Regularization Techniques

- **L1 and L2 Regularization:** Penalizing large weights in the loss function.
- **Dropout:** Randomly setting weights to zero during training to avoid overfitting.
- **Batch Normalization:** Normalizing activations in each layer to improve convergence.

5.2 Optimizers in Keras

- **Stochastic Gradient Descent (SGD):** Classic gradient descent and variants.
- **Adam Optimizer:** Adaptive moment estimation for faster convergence.
- **RMSProp:** Root Mean Square Propagation for better performance.
- **Learning Rate Schedulers:** Adjusting the learning rate dynamically.

6. Advanced Models in Keras

6.1 Convolutional Neural Networks (CNNs) in Keras

- **Understanding CNN Layers:**
 - Convolutional layers: Extracting features from images.
 - Pooling layers: Reducing spatial dimensions (MaxPool, AveragePool).
 - Flattening: Preparing data for fully connected layers.
- **Building CNN Models:** For image classification, object detection.
- **Transfer Learning with CNNs:** Using pre-trained CNN models like VGG, ResNet, and Inception for fine-tuning.

6.2 Recurrent Neural Networks (RNNs) in Keras

- **Understanding RNNs:** Sequence modeling for time-series, text, etc.
- **LSTM and GRU Layers:** Long Short-Term Memory and Gated Recurrent Units for capturing long-term dependencies.
- **Bidirectional RNNs:** Training models on sequences in both directions.
- **Text Generation and Sentiment Analysis:** Using RNNs for natural language processing.

6.3 Generative Models in Keras

- **Autoencoders:** Building autoencoders for dimensionality reduction and feature extraction.
- **Variational Autoencoders (VAEs):** A probabilistic approach to autoencoders for generating new data points.
- **Generative Adversarial Networks (GANs):** Introduction to GANs, building the generator and discriminator networks in Keras.

7. Model Evaluation and Testing

7.1 Model Evaluation Techniques

- **Accuracy:** Measuring the overall correctness of the model.
- **Precision, Recall, and F1-Score:** Evaluating performance based on true positives, false positives, etc.
- **Confusion Matrix:** Visualizing model predictions vs. true labels.
- **ROC and AUC:** Evaluating classification models.

7.2 Model Interpretation

- **Visualizing Weights:** Understanding learned filters in CNNs.
 - **Saliency Maps and Grad-CAM:** Visualizing the regions in an image that influence predictions.
 - **Layer Activations:** Understanding the features learned at different layers of a network.
-

8. Saving and Loading Models

8.1 Saving and Loading Models in Keras

Saving the Entire Model: Saving architecture, weights, and training configuration.

```
model.save('my_model.h5')
```

-

Loading the Model:

```
from keras.models import load_model  
model = load_model('my_model.h5')
```

-

8.2 Saving and Loading Weights Only

Saving Weights: Saving only the model weights.

```
model.save_weights('model_weights.h5')
```

-
- **Loading Weights:** Loading weights into a new model with the same architecture.

8.3 Model Serialization with JSON

Saving Architecture in JSON: Storing the model architecture as JSON.

```
json_string = model.to_json()
```

-

Loading Model from JSON: Rebuilding a model from JSON.

```
from keras.models import model_from_json  
model = model_from_json(json_string)
```

-
-

9. Advanced Keras Topics

9.1 Customizing Training Loop

- **Custom Loss Functions:** Writing your own loss functions for specific tasks.
- **Custom Metrics:** Defining and using custom metrics to evaluate model performance.
- **Custom Layers:** Creating your own neural network layers.

9.2 Keras with TensorFlow 2.x

- **TensorFlow 2.x Integration:** Using Keras within the TensorFlow 2.x ecosystem.
- **TensorFlow Datasets:** Loading and preprocessing datasets with TensorFlow.
- **Model Deployment with TensorFlow Serving:** Deploying Keras models for inference.

9.3 Distributed Training with Keras

- **Multi-GPU Training:** Distributing training across multiple GPUs.
 - **Data Parallelism:** Splitting data across devices.
 - **TensorFlow Distributed Training:** Using Keras for large-scale training.
-

10. Model Deployment and Production

10.1 Deploying Keras Models

- **Serving Keras Models:** Using Flask or FastAPI to serve a Keras model in production.
- **Deploying on Cloud Platforms:** Deploying Keras models to cloud services like AWS, GCP, or Azure.

10.2 Monitoring and Maintaining Models in Production

- **Model Drift:** Detecting when model performance degrades over time.
 - **Model Retraining:** Setting up systems to retrain models with new data.
-

Syllabus Summary:

1. **Introduction to Keras:** Overview, installation, and high-level API structure.
2. **Basic Concepts:** Tensors, layers, and model architecture (Sequential and Functional).
3. **Building Deep Models:** Training, compiling, and evaluating models.
4. **Advanced Architectures:** CNNs, RNNs, GANs, Autoencoders.

5. **Model Regularization and Optimization:** Techniques to improve performance.
6. **Evaluation and Hyperparameter Tuning:** Metrics, validation, and grid search.
7. **Model Saving and Deployment:** Saving models, using models for inference.
8. **Advanced Keras Features:** Custom layers, training loops, distributed training.
9. **Deployment and Monitoring:** Serving models and maintaining them in production.